

Session 5.2

Meaning in Vectors: Embeddings, Attention, the Transformer

Embed, then contextualize: from one vector per word to one vector per word in its sentence

Based on Chapter 13 of Miao & Feng, *AI for Economists*

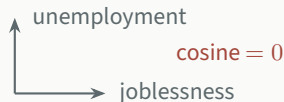


Zhigang Feng

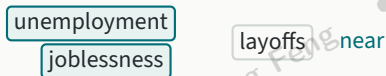
Where the Pipeline Stands



counting: one axis per word



embedding: one point per word

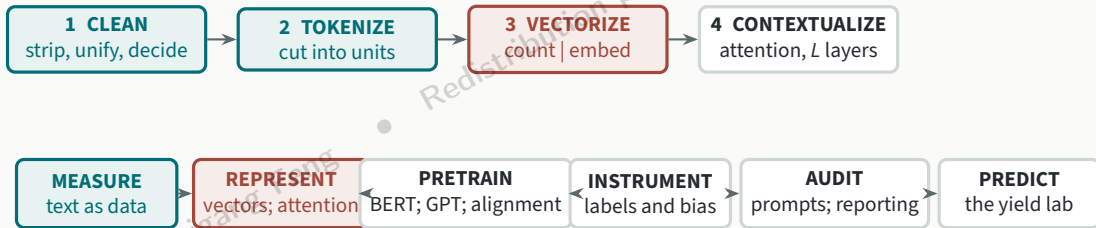


- ▶ Deck 5.1 cleaned the text, cut it into **tokens** (the units you count) and counted them: each document became a **vector**, one entry per vocabulary word.
- ▶ Every word is its own axis, so two different words have inner product 0 and **cosine** (the cosine of their angle) 0: *unemployment* and *joblessness*, *hike* and *raise*.
- ▶ **The repair: make words points, where nearness means similar meaning.**

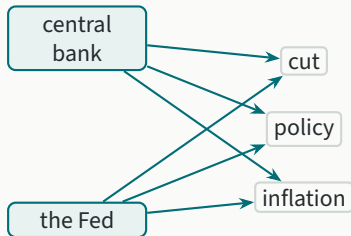
Session 5.2

Step 3 • Embed

Words become points, and closeness becomes meaning



Know a Word by the Company It Keeps



two words, one set of neighbours:
their vectors land close

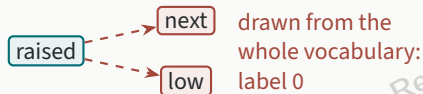
- ▶ Firth (1957): “You shall know a word by the company it keeps.” This is the **distributional hypothesis**.
- ▶ *Central bank* and *Fed* both appear near *cut*, *policy* and *inflation*. Nobody declared them related; their contexts did.
- ▶ A **distributed representation** gives each word w a dense vector $\mathbf{u}_w \in \mathbb{R}^d$ with d far below the vocabulary size V , learned from co-occurrence. Its axes carry no names.
- ▶ **Similar context profiles, nearby vectors: the orthogonality of counting is gone.**

Word2Vec, Step 1: A Window Turns Text Into Pairs

$\Delta = 2$ around the centre word



four real pairs: label 1

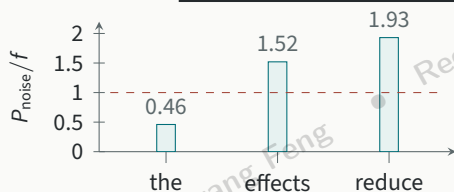


$K = 2$ noise words for the pair (raised, central)

- ▶ Each token in turn is the **centre word**; tokens within Δ positions are its **context words**: each pair is real, label 1.
- ▶ Per real pair, K **noise words** are drawn from the whole vocabulary and paired with the same centre: label 0.
- ▶ $P_{\text{noise}}(w) = f(w)^{3/4} / \sum_v f(v)^{3/4}$, $f(w)$ = w 's share of the tokens. By f , the commonest words dominate; uniform, rare ones do; $3/4$ worked best (Mikolov et al. 2013).
- ▶ **The corpus supplies the data: real pairs against random ones.**

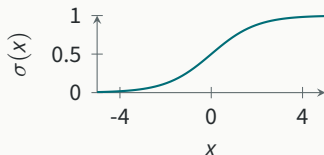
Example: The Training Rows for One Sentence

real pair	label	its $K = 2$ noise pairs, drawn at random	label
(raised, central)	1	(raised, next), (raised, low)	0
(raised, bank)	1	(raised, lending), (raised, 3)	0
(raised, interest)	1	(raised, in), (raised, less)	0
(raised, rates)	1	(raised, susan), (raised, earlier)	0



- ▶ Real draws (seed 42) from the 624 words seen ≥ 10 times in the 1994–2014 statements.
- ▶ *the* is 7.34% of their tokens but 3.35% of draws; *reduce*, seen 10 times, is drawn 1.93 times its share.
- ▶ **12 labelled rows from one centre word, every label free.**

Word2Vec, Step 2: The Objective

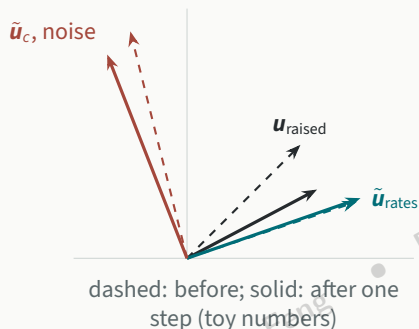


- ▶ $\sigma(x) = 1/(1 + e^{-x})$: $\sigma(\tilde{\mathbf{u}}_c^\top \mathbf{u}_w)$ is the model's probability that (w, c) is a real pair.
- ▶ Each word has a centre vector $\mathbf{u}_w$ and a context vector $\tilde{\mathbf{u}}_w$.

$$\max_{\{\mathbf{u}, \tilde{\mathbf{u}}\}} \sum_{t=1}^T \sum_{\substack{-\Delta \leq j \leq \Delta \\ j \neq 0}} \left[\log \sigma(\tilde{\mathbf{u}}_{w_{t+j}}^\top \mathbf{u}_{w_t}) + \sum_{k=1}^K \log \sigma(-\tilde{\mathbf{u}}_{c_k}^\top \mathbf{u}_{w_t}) \right], \quad c_k \sim P_{\text{noise}}$$

- ▶ Outer sums: every position t of the corpus and every offset j in the window, so every real pair once. Inner sum: that pair's K noise words.
- ▶ A **binary logit** on the last slide's rows: $\log \sigma(x)$ scores a 1, $\log \sigma(-x) = \log(1 - \sigma(x))$ a 0; the vectors are the parameters.
- ▶ **Skip-gram with negative sampling (SGNS): real pairs up, random pairs down.**

Word2Vec, Worked: One Gradient Step Pulls and Pushes



- ▶ A real pair (raised, rates) and a noise pair (raised, c), in 2-D: $x^+ = \tilde{\mathbf{u}}_{\text{rates}}^\top \mathbf{u}_{\text{raised}} = 2.0$, $\sigma(x^+) = 0.88$; $x^- = 1.5$, $\sigma(-x^-) = 0.18$.
- ▶ Gradient for the centre:
 $(1 - \sigma(x^+)) \tilde{\mathbf{u}}_{\text{rates}} - \sigma(x^-) \tilde{\mathbf{u}}_c$, a pull of 0.12, a push of 0.82. The context vectors move too.
- ▶ One step, $\eta = 0.25$: $x^+ 2.0 \rightarrow 2.08$, $x^- 1.5 \rightarrow 0.28$; objective $-1.83 \rightarrow -0.96$.
- ▶ **Words with the same contexts get the same pulls, so they end up pointing the same way.**

Window, Dimension, Domain

small window: syntax

quantitative → easing

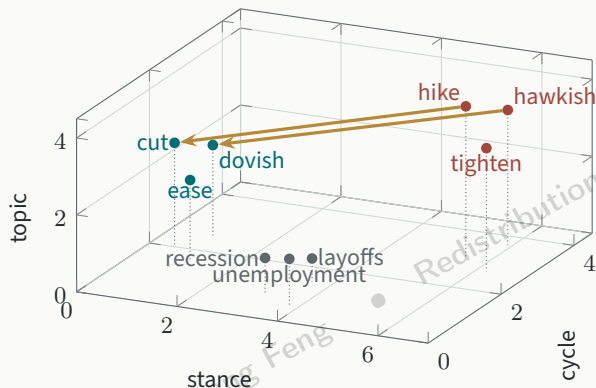
large window: topic

quantitative easing → asset purchases
balance sheet

related by subject, not by grammar

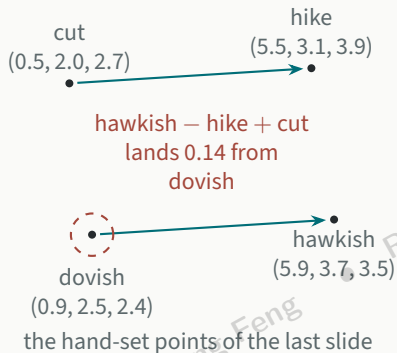
- ▶ A small window ($\Delta \leq 3$) finds syntax; a large one ($\Delta \geq 10$) finds topic, usually what a measure wants. d is typically 100–300.
- ▶ Domain matters more. Trained on general text, *discount* sits near shop markdowns, not the discount rate; *yield* near harvests, not bond yields.
- ▶ CBOW predicts the centre from the averaged context, diluting rare words; GloVe fits $\mathbf{u}_v^\top \tilde{\mathbf{u}}_{v'} + b_v + \tilde{b}_{v'} \approx \log N_{vv'}$ (Pennington et al. 2014).
- ▶ **Train on the corpus you measure.**

The Geometry: Clusters, Parallel Moves, Coupled Axes



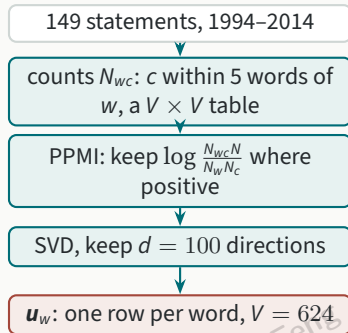
- ▶ Nine points, hand-set on named axes; real vectors have 100–300 unnamed ones.
- ▶ **Clusters:** tightening (red), easing (teal), the cycle (grey).
- ▶ **Parallel moves:** hike→cut and hawkish→dovish point the same way (gold).
- ▶ **Coupling:** *hike* is high on stance *and* topic.
- ▶ **A direction can mean something; an axis rarely does.**

Analogies, Worked, and Why the Metric Is Cosine



- ▶ **Parallelogram rule:** $\mathbf{u}_A - \mathbf{u}_B \approx \mathbf{u}_C - \mathbf{u}_D$.
Here hike - cut = (5.0, 1.1, 1.2) and hawkish - dovish = (5.0, 1.2, 1.1): cosine 0.9996.
- ▶ hawkish - hike + cut = (0.9, 2.6, 2.3), 0.14 from dovish. The famous case: king - man + woman $\approx$ queen.
- ▶ $\|\mathbf{u} - \mathbf{u}'\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{u}'\|^2 - 2\|\mathbf{u}\|\|\mathbf{u}'\|\cos\theta$: distance also reads the lengths, and length tracks frequency.
- ▶ **Compare directions with cosine, not lengths with distance.**

Lab B1: Word Vectors From the FOMC Statements, the Setup



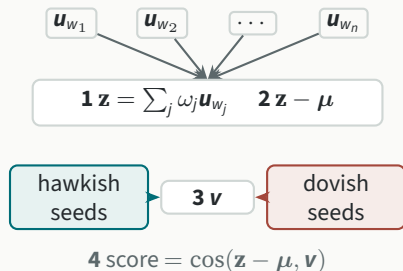
- ▶ The question: what do the Committee's own words mean, judged by their company in its statements? Vectors trained on news would answer for news.
- ▶ 149 statements are too few for skip-gram, so lab cell B1 counts, on words seen ≥ 10 times.
- ▶ PMI: how much more often w and c meet than chance (N : all pairs). Then $\text{PPMI} \approx \mathbf{U}_d \Sigma_d \mathbf{V}_d^\top$; $\mathbf{u}_w = \text{row } w \text{ of } \mathbf{U}_d \Sigma_d^{1/2}$, length 1.
- ▶ **A close cousin of SGNS, whose optimum has $\tilde{\mathbf{u}}_c^\top \mathbf{u}_w = \text{PMI}(w, c) - \log K$ (Levy & Goldberg 2014).**

What the FOMC Vectors Learned

word	its five nearest neighbours by cosine				
inflation	expectations	longer	pressures	term	stable
employment	maximum	objectives	seeks	foster	toward
committee	today	decided	open	raise	particular
accommodation	remove	can	removed	begin	take

- ▶ Read the neighbours as phrases: *inflation* meets *expectations*, *longer* and *pressures* in “longer-term inflation expectations” and “inflation pressures”; *maximum employment* is the mandate’s. A 5-word window learns set phrases.
- ▶ “raise : raised :: lower : ?” has no answer: *raised* occurs fewer than 10 times and gets no vector — the method, not the geometry.
- ▶ In B1: print other words’ neighbours; change the window or minimum count; see what survives.
- ▶ **Read the neighbours before trusting a direction.**

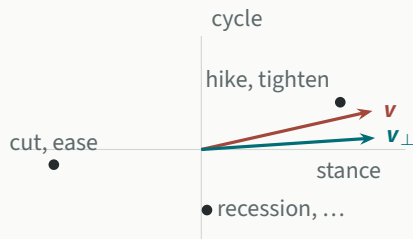
From Word Vectors to a Document Score



document, hand-set points	uncentred	centred
hike, tighten	+0.94	+0.97
cut, ease	+0.54	-0.96

- ▶ Goal: one number per document on a concept (how hawkish?), counting synonyms no list names. ω_j : uniform or TF-IDF, fixed before reading.
- ▶ Centre first: all vectors share a common part, so uncentred even “cut, ease” scores +0.54.
- ▶ Used for emotion vs reason in Congress (Gennaro & Ash 2022), evidence vs intuition (Aroyehun et al. 2025); see Ash & Hansen (2023).
- ▶ **The score is a regressor: validate it like one.**

What a Direction Bundles In



centred documents and directions,
seen on the stance-cycle plane

- ▶ “Recession, unemployment, layoffs” has no policy word, yet scores -0.25 : dovish.
- ▶ Why: hikes are written about in booms, cuts in recessions, so $\mathbf{v} = (5.03, 1.13, 1.13)$ points up the cycle axis too. King and queen differ in one dimension; hike and cut in two.
- ▶ Repair: remove a cycle direction $\mathbf{c}$ (policy minus cycle words), $\mathbf{v}_{\perp} = \mathbf{v} - (\mathbf{v}^{\top} \hat{\mathbf{c}}) \hat{\mathbf{c}}$: scores $+0.88, -1.00, 0.00$.
- ▶ **Remove what you do not mean. The lab’s FOMC hawk–dove score (B2–B3) tracks the policy path at only $+0.07$; centred, $+0.05$.**

Four Limits of One Vector per Word

Polysemy. *rate*: the policy rate, an exchange rate, the inflation rate, a growth rate; *discount*: the discount rate or a price cut. One vector averages every sense.

Corpus dependence. *Mask* sat near face and costume before 2020, near mandate and protection after. A regime shift in language breaks a time series.

Negation. Averaging keeps “growth is slowing” next to “growth has not slowed” — the turn forward guidance is about.

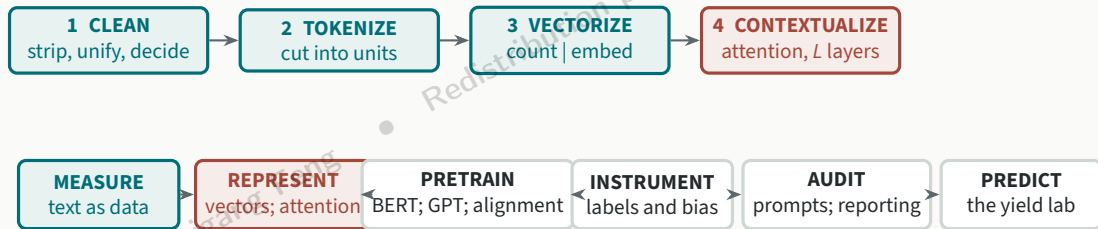
Inherited bias. man : computer programmer :: woman : homemaker (Bolukbasi et al. 2016); association effects at human size (Caliskan et al. 2017). Noise in a feature, a finding as an object.

- All four come from one vector per word, whatever the sentence around it.

Session 5.2

Step 4 • Contextualize

A weight computed from the sentence, instead of fixed before it is read



One Word, Two Sentences

(A) Fed raised policy **rate** inflation

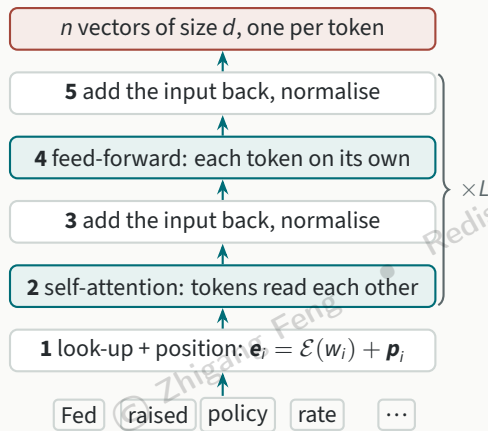
the policy instrument

(B) unemployment **rate** fell

a labour-market statistic

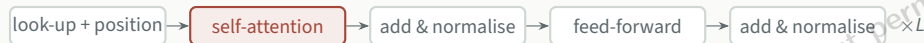
- ▶ (A) The Fed raised its policy *rate* in response to persistent inflation. (B) The unemployment *rate* fell to its lowest level in decades.
- ▶ The same token, *rate*, names two economic objects: an instrument the Fed sets, and a labour-market statistic.
- ▶ A static vector gives both the same $\mathbf{u}_{\text{rate}}$, and a document vector with fixed ω cannot separate them.
- ▶ **The weights must be computed from the sentence itself.**

The Transformer Layer at a Glance



- In: n tokens. Out: n vectors of size d , each now carrying its sentence.
- Only step 2 moves information between positions; the others work on one token's vector.
- Steps 3 and 5 keep each vector and rescale it; step 4 transforms it with one small shared network.
- **Steps 2–5 repeat L times (12 in multilingual BERT). The next frames open each box.**

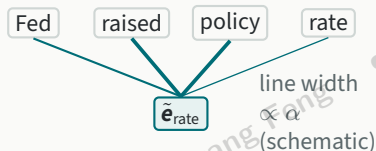
The Idea: Re-Weight the Sentence for Each Word



embedding table $\mathcal{E}$: one row per token ID

... rate

... raised



- ▶ The **embedding table** $\mathcal{E}$ holds one learned vector per token ID; add a position vector: $\mathbf{e}_i = \mathcal{E}(w_i) + \mathbf{p}_i$.
- ▶ Rebuild token i from the sentence: $\tilde{\mathbf{e}}_i = \sum_j \alpha_{ij} \mathcal{E}(w_j)$, $\alpha_{ij} \geq 0$, $\sum_j \alpha_{ij} = 1$. The next frames refine this.
- ▶ Unlike ω , the α_{ij} come from *this* sentence: in (A) *rate* leans on raised and policy; in (B) on unemployment and fell.
- ▶ **The same token, different neighbours read, a different vector: self-attention.**

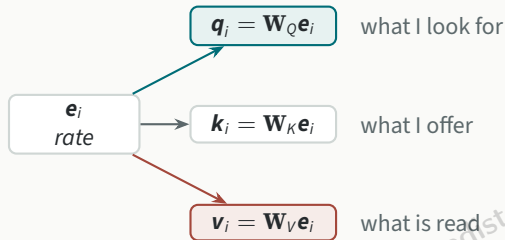
Attention Began as Alignment in Translation

	央行	上调	利率
The	0.70	0.15	0.15
central bank	0.85	0.10	0.05
raised	0.10	0.80	0.10
rates	0.05	0.15	0.80

rows: the English word being written; columns: the Chinese source (central bank, raised, interest rate). Illustrative weights

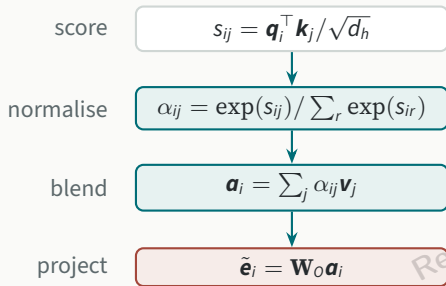
- ▶ A recurrent network reads one token at a time into one state; translating, it squeezed the whole source into it.
- ▶ Attention let each word being written look back at every source word (Bahdanau et al. 2015): *raised* reads 上调, *rates* reads 利率. This is **cross-attention**.
- ▶ The Transformer drops the recurrence (Vaswani et al. 2017): it trains in parallel, and any two positions are one step apart.
- ▶ **Self-attention is this alignment, turned on one sentence.**

Query, Key, Value: Three Roles for One Token



- ▶ Three learned matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_h \times d}$ give each token three roles — request, index label, content: a soft database look-up.
- ▶ In (A), the query of *rate* meets the keys of Fed, raised, policy and inflation.
- ▶ In “The discount rates applied to pension funds were revised”, *rates* meets discount and pension instead.
- ▶ **Three separate matrices, so attention is asymmetric: i may read j heavily while j barely reads i .**

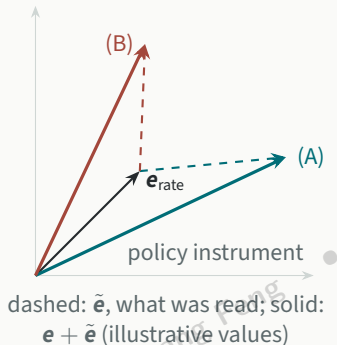
Score, Normalise, Blend, Project



- Sizes: $\mathbf{q}_i, \mathbf{k}_j, \mathbf{v}_j \in \mathbb{R}^{d_h}$; s_{ij} is one number; $\mathbf{a}_i \in \mathbb{R}^{d_h}$; $\mathbf{W}_O \in \mathbb{R}^{d \times d_h}$, so $\tilde{\mathbf{e}}_i \in \mathbb{R}^d$, the size of $\mathbf{e}_i$.
- The softmax makes each row a distribution: position i spends one unit of attention, on the value vectors.
- Why $\sqrt{d_h}$: the variance of $\mathbf{q}^\top \mathbf{k}$ grows with the head width d_h . Unscaled, the softmax piles onto one position and the gradients elsewhere vanish.
- $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$ and $\mathcal{E}$ are learned; α_{ij} is computed for every input: a share, never a coefficient.

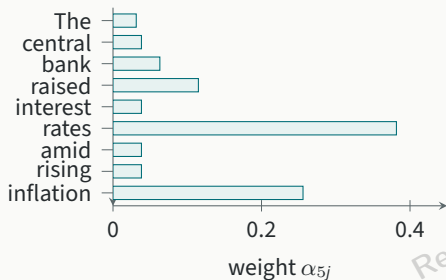
From $\mathbf{e}_i$ to $\tilde{\mathbf{e}}_i$: Same Size, New Content

labour statistic



- ▶ $\mathbf{e}_i$: the token alone, the same in every sentence (for *rate*, between senses).
- ▶ $\tilde{\mathbf{e}}_i$: what it read, a weighted mix of other tokens' values, also of size d , so it adds: $\mathbf{e}_i + \tilde{\mathbf{e}}_i$.
- ▶ One piece of information it carries: *which kind of rate*. (A) reads Fed, raised, policy, inflation: $\tilde{\mathbf{e}} = (0.84, 0.08)$; (B) unemployment, fell, lowest, decades: $(0.03, 0.73)$.
- ▶ **Same size, new content: the two *rate* vectors, identical before, now have cosine 0.77.**

Example 1: Scores to Weights for the Query *interest*



token j	s_{5j}	$e^{s_{5j}}$	α_{5j}
The	-0.2	0.82	0.03
central	0	1.00	0.04
bank	0.5	1.65	0.06
raised	1.1	3.00	0.11
interest	0	1.00	0.04
rates	2.3	9.97	0.38
amid	0	1.00	0.04
rising	0	1.00	0.04
inflation	1.9	6.69	0.26

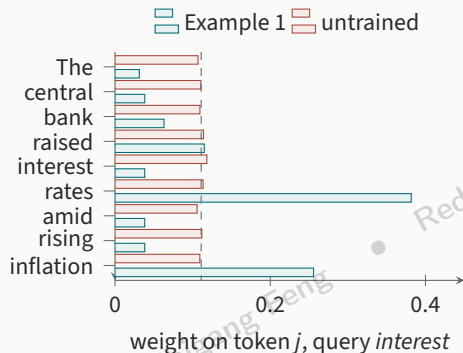
- ▶ Query *interest*. Trained, $s_{5j} = \mathbf{q}_5^\top \mathbf{k}_j / \sqrt{d_h}$; here set for illustration. $\alpha_{5j} = e^{s_{5j}} / \sum_r e^{s_{5r}}$, the sum 26.13.
- ▶ **64% of the attention goes to *rates* and *inflation*: this *interest* is an interest rate, not a stake.**

Example 2: Blend, Project, Add

		level	up	prices
rates	$0.38 \times$	1	0.3	0
inflation	$0.26 \times$	0	0.2	1
raised	$0.11 \times$	0	1	0
rising	$0.04 \times$	0	0.5	0.2
bank	$0.06 \times$	0.5	0	0
$+ \dots = \mathbf{a}_5$		0.46	0.30	0.26
$\mathbf{e}_{\text{interest}}$		1	0	0
$\mathbf{e} + \tilde{\mathbf{e}}$		1.46	0.30	0.26

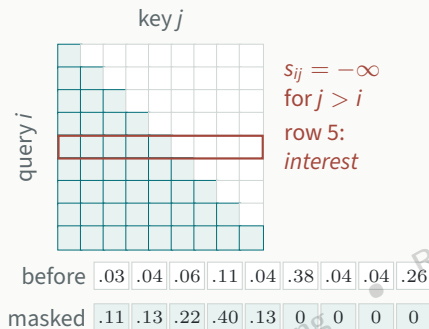
- ▶ Each token's value vector gets three named coordinates: rate level, direction up, price pressure (illustrative; real ones are unnamed).
- ▶ Blend with Example 1's weights:
 $\mathbf{a}_5 = \sum_j \alpha_{5j} \mathbf{v}_j = (0.46, 0.30, 0.26)$;
with $\mathbf{W}_O = \mathbf{I}$ here, $\tilde{\mathbf{e}}_5 = \mathbf{a}_5$.
- ▶ Add the token's own vector:
 $(1.0, 0.0, 0.0) + (0.46, 0.30, 0.26) = (1.46, 0.30, 0.26)$.
- ▶ **Same size, new content: *interest* now says “going up” and “price pressure”.**

Example 3: The Same Arithmetic, Untrained



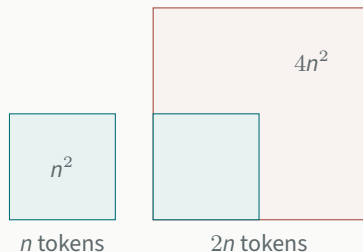
- ▶ The same tokens through an untrained head: random embeddings ($d = 16$) and random $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ ($d_h = 8$).
- ▶ Its row for *interest*: 0.106–0.118, against $1/9 = 0.111$. Nearly flat.
- ▶ Training (Deck 5.3) fits $\mathbf{W}_Q, \mathbf{W}_K$ so that the keys that matter score high. Lab C1 reproduces Example 1; C1b, this head.
- ▶ **The arithmetic is fixed; the pattern is learned.**

The Causal Mask: A Writer Must Not Read Ahead



- ▶ Goal: a generator writes token $i + 1$ from tokens $1, \dots, i$ (Deck 5.3). Trained on whole sentences at once, unmasked, position i could read the answer, token $i + 1$, and copy it.
- ▶ Set $s_{ij} = -\infty$ for $j > i$: then $\alpha_{ij} = 0$ and each row renormalises over $j \leq i$. This is the **causal mask**.
- ▶ Example 1 masked: the sum falls to 7.47; *raised* takes 0.40; *rates* and *inflation* get 0. Lab C1b masks the untrained head.
- ▶ **An encoder keeps the square; a decoder uses the triangle.**

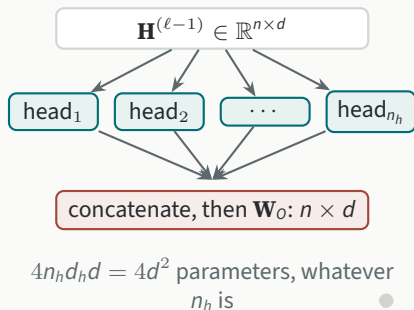
What the Square Costs



text	tokens n	scores n^2
the nine-token sentence	9	81
median FOMC statement	410	168,100
longest statement	1,053	1,108,809
100 statements	41,000	1,681,000,000

- ▶ Every query scores every key: n tokens give n^2 scores in each head of each layer, stored as an $n \times n$ table. (Counts: the lab's 245 statements, GPT-4's tokenizer.)
- ▶ Doubling n quadruples work and memory; 100 times longer costs 10,000 times as much.
- ▶ So a long filing is cut into blocks that cannot see each other, severing a cross-reference from Item 7 back to Item 1A: Deck 5.3's window limit.
- ▶ **The square is the price of letting every token read every other.**

Many Heads, and the Whole Sentence at Once

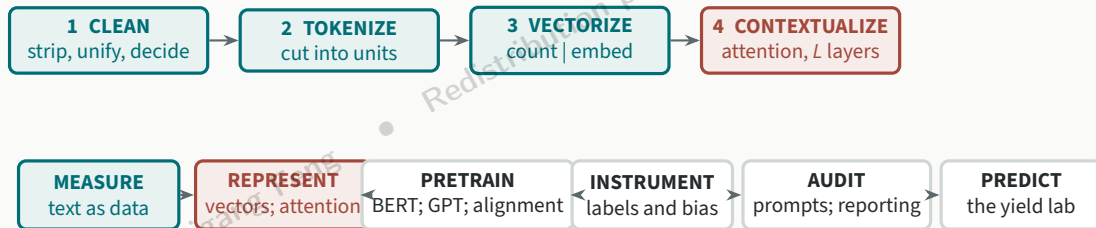


- ▶ Stack the tokens as rows, $\mathbf{H}^{(\ell-1)} \in \mathbb{R}^{n \times d}$. Each head computes $\text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_h}) \mathbf{V}$ at width $d_h = d/n_h$; the output is again $n \times d$.
- ▶ Heads are like readers of one set of minutes: one follows inflation language, one the labour market, one references back such as “since the Committee’s last meeting”.
- ▶ More heads redistribute capacity; they add no parameters. Nothing assigns their jobs.
- ▶ **A head weights positions; it does not split a document into topics.**

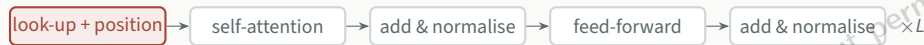
Session 5.2

One Transformer Layer

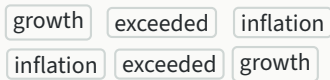
Position, residual connections, normalisation, and the feed-forward block



Attention Cannot See Word Order



the same tokens, opposite claims

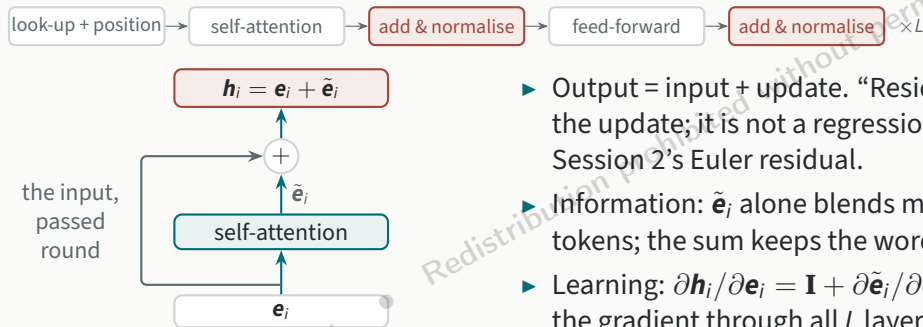


queries and keys see tokens, not places

$$\mathbf{e}_i = \mathcal{E}(w_i) + \mathbf{p}_i$$

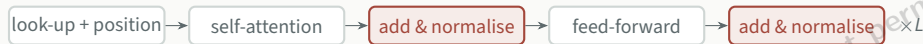
- ▶ Without position vectors, attention is permutation-invariant: shuffle the tokens and no score changes (a causal mask leaks some order; Haviv et al. 2022).
- ▶ **Sinusoidal:** $p_{i,2l} = \sin(i/10^{8l/d})$, $p_{i,2l+1} = \cos(i/10^{8l/d})$.
- ▶ Learned positions stop at the training length; RoPE (Su et al. 2024) with scaling and ALiBi (Press et al. 2022) read past it.
- ▶ **Order is not in the mechanism; it is added to the input.**

Residual Connection: Keep the Token, Add What It Read



- ▶ Output = input + update. “Residual” names the update; it is not a regression residual or Session 2’s Euler residual.
- ▶ Information: $\tilde{e}_i$ alone blends mostly other tokens; the sum keeps the word too.
- ▶ Learning: $\partial h_i / \partial e_i = \mathbf{I} + \partial \tilde{e}_i / \partial e_i$. The $\mathbf{I}$ carries the gradient through all L layers; without it, one Jacobian per layer shrinks it: the **vanishing gradient**.
- ▶ A layer with nothing to add learns $\tilde{e}_i \approx 0$ and passes its input on.

Layer Normalisation: Standardise Each Token's Vector



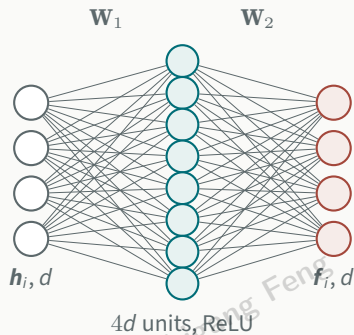
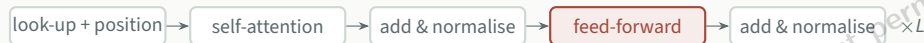
one token's vector, $d = 4$

2	4	6	8
$(\mathbf{x} - \mu) / \sigma$ $\mu = 5, \sigma = 2.24$			
-1.34	-0.45	0.45	1.34

then $\gamma \odot (\cdot) + \beta$, learned per coordinate

- ▶ $\text{LN}(\mathbf{x}) = \gamma \odot (\mathbf{x} - \mu) / \sqrt{\sigma^2 + \epsilon} + \beta$: μ, σ = mean and s.d. of the d entries of *one* token's vector; $\gamma, \beta \in \mathbb{R}^d$ learned; ϵ tiny.
- ▶ Why: each residual adds to the vector, so its scale drifts from layer to layer; LN puts each layer's input on one scale, like standardised regressors.
- ▶ Originally applied after the addition; Machine B and most current models apply it before each block.
- ▶ **Neither the residual nor LN moves information between tokens. Only attention does.**

The Feed-Forward Block: Transform Each Token on Its Own



- ▶ Attention only averages values, a linear mix. Combining features — *rate, raised, inflation* into “tightening” — needs a nonlinear step.
- ▶ $f_i = W_2 \text{ReLU}(W_1 h_i + b_1) + b_2$, $W_1 \in \mathbb{R}^{4d \times d}$, $W_2 \in \mathbb{R}^{d \times 4d}$, $\text{ReLU}(x) = \max(0, x)$: Session 2’s one-hidden-layer network.
- ▶ Same weights at every position; output size d , added back. Machine B uses GELU, a smooth ReLU.
- ▶ **Attention moves information between tokens; this block transforms each one.**

Example: A Hidden Unit as a Key-Value Memory

	level	up	prices	tight	
h	1.46	0.30	0.26	0.00	
key 1	1	1	1	0	→ 0.52
value 1	0	0	0	1	
key 2	1	-1	0	0	→ 0.00
$h + f$	1.46	0.30	0.26	0.52	●

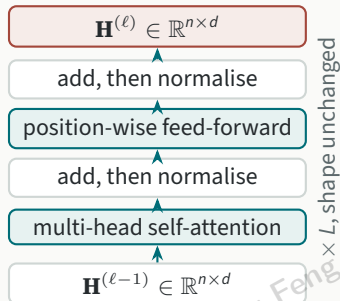
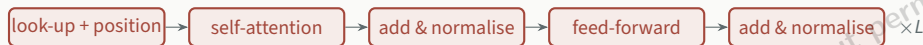
- ▶ Illustrative: coordinates rate level, up, price pressure, tightening; h from Example 2; bias -1.5 . Unit 1's key (a row of W_1): $2.02 + (-1.5) = 0.52$. It fires on “a rate going up with price pressure” and adds $0.52 \times$ its value (a column of W_2).
- ▶ Unit 2 looks for “a rate going down”: -0.34 , ReLU 0, silent. The vector now says “tightening”, which no token said.
- ▶ **Rows of W_1 detect patterns; columns of W_2 write back what goes with them. Trained keys respond to recognisable text patterns (Geva et al. 2021).**

Where the Parameters Are: Machine B, Part by Part

part of Machine B	count	share
feed-forward $\mathbf{W}_1$, 3 layers	198,144	31.4%
feed-forward $\mathbf{W}_2$, 3 layers	196,992	31.2%
attention $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$, 3 layers	148,608	23.5%
attention $\mathbf{W}_O$, 3 layers	49,536	7.8%
position table, 128 positions	16,384	2.6%
character table, 78 characters	9,984	1.6%
output layer, 78 characters	9,984	1.6%
layer norms before attention	768	0.1%
layer norms before feed-forward	768	0.1%
final layer norm	256	0.0%
whole model	631,424	100%

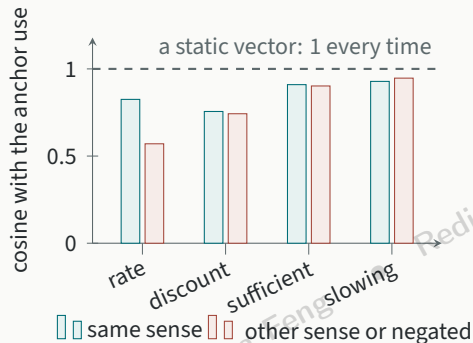
- **Machine B:** Session 1's character-level text generator, trained on the 245 FOMC statements less every 20th; 3 layers, 4 heads, $d = 128$.
- Per layer, feed-forward $8d^2$ against attention's $4d^2$: 62.6% of the model. Lab C2 prints this table.
- **Most parameters sit in the feed-forward blocks, where associations learned from text are kept.**

One Layer, Stacked L Times



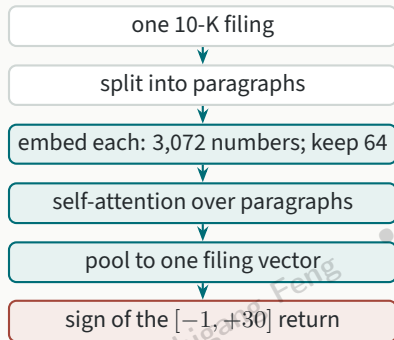
- ▶ The layer's output has the input's shape, $n \times d$, so layers stack with no change of interface.
- ▶ Shallow layers lean to local syntax (*quantitative easing*); deep ones link paragraphs, a “risk factor” early in an annual report with a “regulatory change” later.
- ▶ Machine B stacks 3 layers, $d = 128$; multilingual BERT 12, $d = 768$.
- ▶ **$\mathbf{H}^{(L)}$ is where Deck 5.3 starts.**

What Context Buys, Measured



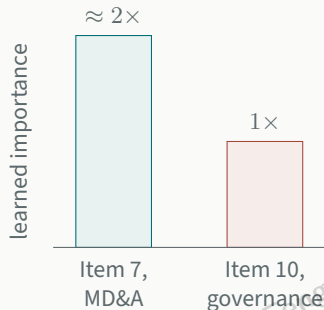
- ▶ Multilingual BERT (12 layers, $d = 768$), last layer: one word in three sentences, cosine with its anchor use.
- ▶ *rate* separates its senses, 0.825 vs 0.570 (unemployment rate); *discount* barely, 0.756 vs 0.743 (a price discount).
- ▶ Negation barely moves it: *sufficient* 0.910 vs 0.902; for *slowing* the negated use is the closer, 0.947 vs 0.928.
- ▶ **Context can fix a sense; whether it does, and whether it reads “not”, must be measured.**

Reading 10-Ks by Paragraph Attention: The Design



- ▶ Attention is not tied to word tokens: Kim, Muhn, Nikolaev & Zhang (2024) embed each paragraph as one vector, so the paragraph is the “token”.
- ▶ 76,929 filings from 2004 to 2024, split into 20,712,462 paragraphs.
- ▶ **The target is a direction: is the cumulative return from one day before the filing to thirty days after positive?**

What the Learned Weights Say, and What They Cannot



- ▶ Attention beats both attention-free baselines (Kim et al. 2024).
- ▶ MD&A is attended most, almost twice as much as governance.
- ▶ Learned to predict returns: “important” means useful, not causal. In a sentence, too, α_{ij} is an intermediate quantity, and boilerplate repeated in every filing may spread it thin.
- ▶ **Attention routes information inside the text it is given. It does not retrieve truth.**

The Whole Pipeline

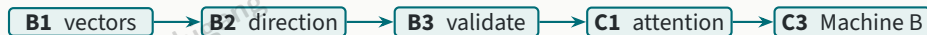


rung	“rates” becomes	size	similar words	senses	you can read
counts, TF-IDF	one column of the matrix	$V \sim 10^4\text{--}10^5$, mostly zeros	orthogonal	one per word	each axis is a word
word vector	one point $\mathbf{u} \in \mathbb{R}^d$	$d = 100\text{--}300$, dense	close	one per word	no axis is a word
contextual vector	a point per sentence	● $d = 768$ in multilingual BERT	close	one per use	the weights, not the meaning

- ▶ Report every step: the cleaning script; the tokenizer and its version; the vector method, corpus and period; the model and its version.
- ▶ **Each rung buys meaning and sells the audit chain: pick the rung the design needs.**

Lab, Parts B and C: Vectors and Attention on the FOMC Statements

cell	what you run	the question you answer
B1	word vectors by PPMI and SVD	Which neighbours are phrases? Which survive another window?
B2	the hawk–dove direction	Which seeds survive, and what does the score bundle in?
B3	validation on the policy path	Where does the embedding score fail?
C1, C1b	Example 1; the untrained head, masked	What does the mask set to 0? How flat is the row?
C2–C3	Machine B's parameters and maps	Where are the parameters? Is the upper triangle 0?



- **Each cell reproduces the slide numbers it names; C4–C5 belong to Deck 5.3.**